User Manual

YESENSE Communication Protocol Description Document

Document Version 2.3, 21 October 2023

Catalogue

1 Overview **2**

2 Introduction **3**

2.1 Serial port output protocol 3

2.1.1 Frame format 3

2.1.2 Payload data definition 3

The definition of payload is shown in the following table: 3

2.1.3 Checksum 8

2.2 Serial command protocol 9

2.2.1 Frame format 9

2.2.2 Payload data definition 9

2.2.3 Checksum 14

3 Reference **15**

3.1 Examples 15

3.1.1 Example message of serial output protocol 15

3.1.2 Command examples 17

4 Revisions **19**

1 Overview

This document describes YESENSE private protocols for integrating and configuring YESENSE products.

The YESENSE private protocol specification describes the communication data format used by the YESENSE products, including two parts of output data frames and configuration data frames.

2 Introduction

The YESENSE private protocol is designed for communicating with the host machine. The protocol has the following features:

1) Compact: 8bit binary data

2) Data verification: using a low-overhead verification algorithm

3) Unification: all multi-byte data is in the little-endian mode

2.1 Serial port output protocol

2.1.1 Frame format

The YESNESE private serial port output protocol adopts a fixed format and the frame structure includes five parts: header, frame sequence number, payload data length, payload data field and checksum, as shown in the table below.

identifier	length	description
header	2 bytes	frame header is 0x59, 0x53 in little-endian
frame sequence number	2 bytes	range is from 1 to 60000
payload data length	1 byte	range is from 0 to 255
payload data field	from 0 to 255 bytes	valid data of outputted message
first checksum	1 byte	checksum
second checksum	1 byte	

2.1.2 Payload data definition

The definition of payload is shown in the following table:

data message 1			……	data message N		
DATA ID	LEN	DATA(LEN bytes)	……	DATA ID	LEN	DATA(LEN bytes)

Data in payload field can be composed of multiple combinations of data message. Each different packet contains a specific identifier called DATA ID, the corresponding data length indicator called LEN and data content of the message called DATA.

Different messages are defined in following table.

message type	DATA ID	LEN	DATA
sensor's temperature	0x01	2 bytes	DATA1 – DATA2
acceleration	0x10	12 bytes	DATA1 – DATA12
angular velocity	0x20	12 bytes	DATA1 – DATA12
magnetic field normalized value	0x30	12 bytes	DATA1 – DATA12
magnetic field raw value	0x31	12 bytes	DATA1 – DATA12
euler angle	0x40	12 bytes	DATA1 – DATA12

quaternion	0x41	16 bytes	DATA1 – DATA16
utc time	0x50	11 bytes	DATA1 – DATA11
sampling timestamp	0x51	4 bytes	DATA1 – DATA4
dataready timestamp	0x52	4 bytes	DATA1 – DATA4
position	0x68	20 bytes	DATA1 – DATA20
velocity	0x70	12 bytes	DATA1 – DATA12
combined navigation status	0x80	1 byte	DATA1

All the above data are signed data, except utc time, sampling timestamp, dataready timestamp and combined navigation status.

The conversion relationship from output message to real data value is shown in the following table.

message type	DATA	data conversion	unit
sensor's temperature	DATA1（DATA[7:0]）	temperature = DATA ×0.01	℃
	DATA2（DATA[15:8]）		
acceleration	DATA1（DATA[7:0]）	acceleration of x axis = DATA × 0.000001	m/s²
	DATA2（DATA[15:8]）		
	DATA3（DATA[23:16]）		
	DATA4（DATA[31:24]）		
	DATA5（DATA[7:0]）	acceleration of y axis = DATA × 0.000001	
	DATA6（DATA[15:8]）		
	DATA7（DATA[23:16]）		
	DATA8（DATA[31:24]）		
	DATA9（DATA[7:0]）	acceleration of z axis = DATA × 0.000001	
	DATA10（DATA[15:8]）		
	DATA11（DATA[23:16]）		
	DATA12（DATA[31:24]）		
angular velocity	DATA1（DATA[7:0]）	angular velocity of x axis = DATA × 0.000001	deg/s
	DATA2（DATA[15:8]）		
	DATA3（DATA[23:16]）		
	DATA4（DATA[31:24]）		
	DATA5（DATA[7:0]）	angular velocity of y axis = DATA × 0.000001	
	DATA6（DATA[15:8]）		
	DATA7（DATA[23:16]）		
	DATA8（DATA[31:24]）		
	DATA9（DATA[7:0]）	angular velocity of z axis = DATA × 0.000001	
	DATA10（DATA[15:8]）		
	DATA11（DATA[23:16]）		

	DATA12（DATA[31:24]）		
magnetic field normalized values	DATA1（DATA[7:0]）	normalized value of x axis = DATA × 0.000001	none
	DATA2（DATA[15:8]）		
	DATA3（DATA[23:16]）		
	DATA4（DATA[31:24]）		
	DATA5（DATA[7:0]）	normalized value of y axis = DATA ×0.000001	
	DATA6（DATA[15:8]）		
	DATA7（DATA[23:16]）		
	DATA8（DATA[31:24]）		
	DATA9（DATA[7:0]）	normalized value of z axis = DATA ×0.000001	
	DATA10（DATA[15:8]）		
	DATA11（DATA[23:16]）		
	DATA12（DATA[31:24]）		
magnetic field raw values	DATA1（DATA[7:0]）	normalized value of x axis = DATA × 0.001	mGauss
	DATA2（DATA[15:8]）		
	DATA3（DATA[23:16]）		
	DATA4（DATA[31:24]）		
	DATA5（DATA[7:0]）	normalized value of y axis = DATA × 0.001	
	DATA6（DATA[15:8]）		
	DATA7（DATA[23:16]）		
	DATA8（DATA[31:24]）		
	DATA9（DATA[7:0]）	normalized value of z axis = DATA × 0.001	
	DATA10（DATA[15:8]）		
	DATA11（DATA[23:16]）		
	DATA12（DATA[31:24]）		
euler angle	DATA1（DATA[7:0]）	pitch = DATA × 0.000001	deg(°)
	DATA2（DATA[15:8]）		
	DATA3（DATA[23:16]）		
	DATA4（DATA[31:24]）		
	DATA5（DATA[7:0]）	roll = DATA × 0.000001	
	DATA6（DATA[15:8]）		
	DATA7（DATA[23:16]）		
	DATA8（DATA[31:24]）		
	DATA9（DATA[7:0]）	yaw = DATA × 0.000001	
	DATA10（DATA[15:8]）		
	DATA11（DATA[23:16]）		
	DATA12（DATA[31:24]）		
quaternion	DATA1（DATA[7:0]）	quaternion0 = DATA × 0.000001	

	DATA2（DATA[15:8]）		
	DATA3（DATA[23:16]）		
	DATA4（DATA[31:24]）		
	DATA5（DATA[7:0]）	quaternion1 = DATA × 0.000001	
	DATA6（DATA[15:8]）		
	DATA7（DATA[23:16]）		
	DATA8（DATA[31:24]）		
	DATA9（DATA[7:0]）	quaternion2 = DATA × 0.000001	
	DATA10（DATA[15:8]）		
	DATA11（DATA[23:16]）		
	DATA12（DATA[31:24]）		
	DATA13（DATA[7:0]）	quaternion3 = DATA × 0.000001	
	DATA14（DATA[15:8]）		
	DATA15（DATA[23:16]）		
	DATA16（DATA[31:24]）		
utc time	DATA1（DATA[7:0]）	millisecond = DATA	ms
	DATA2（DATA[15:8]）		
	DATA3（DATA[23:16]）		
	DATA4（DATA[31:24]）		
	DATA5（DATA[7:0]）	year = DATA	year
	DATA6（DATA[15:8]）		
	DATA7（DATA[7:0]）	month = DATA	month
	DATA8（DATA[7:0]）	day = DATA	day
	DATA9（DATA[7:0]）	hour = DATA	hour
	DATA10（DATA[7:0]）	minute = DATA	minute
	DATA11（DATA[7:0]）	second = DATA	second
sampling timestamp	DATA1（DATA[7:0]）	sample timestamp = DATA	us
	DATA2（DATA[15:8]）		
	DATA3（DATA[23:16]）		
	DATA4（DATA[31:24]）		
dataready timestamp	DATA1（DATA[7:0]）	dataready timestamp = DATA	us
	DATA2（DATA[15:8]）		
	DATA3（DATA[15:8]）		
	DATA4（DATA[31:24]）		
position	DATA1（DATA[7:0]）	latitude = DATA * 0.0000000001	degree
	DATA2（DATA[15:8]）		
	DATA3（DATA[23:16]）		
	DATA4（DATA[31:24]）		

	DATA5（DATA[39:32]）		
	DATA6（DATA[47:40]）		
	DATA7（DATA[55:48]）		
	DATA8（DATA[63:56]）		
	DATA9（DATA[7:0]）	longitude = DATA × 0.0000000001	degree
	DATA10（DATA[15:8]）		
	DATA11（DATA[23:16]）		
	DATA12（DATA[31:24]）		
	DATA13（DATA[39:32]）		
	DATA14（DATA[47:40]）		
	DATA15（DATA[55:48]）		
	DATA16（DATA[63:56]）		
	DATA17（DATA[63:56]）	altitude = DATA × 0.001	m
	DATA18（DATA[7:0]）		
	DATA19（DATA[23:16]）		
	DATA20（DATA[31:24]）		
velocity	DATA1（DATA[7:0]）	velocity in the e direction = DATA * 0.001	m/s
	DATA2（DATA[15:8]）		
	DATA3（DATA[23:16]）		
	DATA4（DATA[31:24]）		
	DATA5（DATA[7:0]）	velocity in the n direction = DATA * 0.001	
	DATA6（DATA[15:8]）		
	DATA7（DATA[23:16]）		
	DATA8（DATA[31:24]）		
	DATA9（DATA[7:0]）	velocity in the u direction = DATA * 0.001	
	DATA10（DATA[15:8]）		
	DATA11（DATA[23:16]）		
	DATA12（DATA[31:24]）		
combined navigation status	DATA1（DATA[3:0]）	fusion_state = DATA1	none
	DATA2（DATA[7:4]）	GNSS_state = DATA2	none

The definition of each part in combined navigation status is explained as the following table.

type	value	description
fusion state	0	0-Uninitialized：No satellite navigation information is received after power, and the default position information is outputted at this time
	1	1-Position initialized: after the satellite navigation information is received, only the original gnss position

		is outputted
	2	2-Heading initialized: the system completes the heading initialization and outputs the accurate heading data
	4	4-Combined GNSS/Dead Reckoning: the system enters the combination fusion state and the outputted messages, including position, speed and euler angle, are calculated by combination of inertial and GNSS
	5	5-Dead Reckoning Only: After the current system enters the state of Combined GNSS/Dead Reckoning, when GNSS is in loss of lock, the system will only calculate navigation information by inertial method
GNSS state	0	No Position Fix
	1	Autonomous GNSS Fix
	2	Differential GNSS Fix
	4	RTK Fixed
	5	RTK Float

2.1.3 Checksum

A whole data frame needs to append checksum to the tail of payload. The calculated range starts from TID to the last byte of Message, as shown in the following figure, and the calculation formula is shown in the following example.

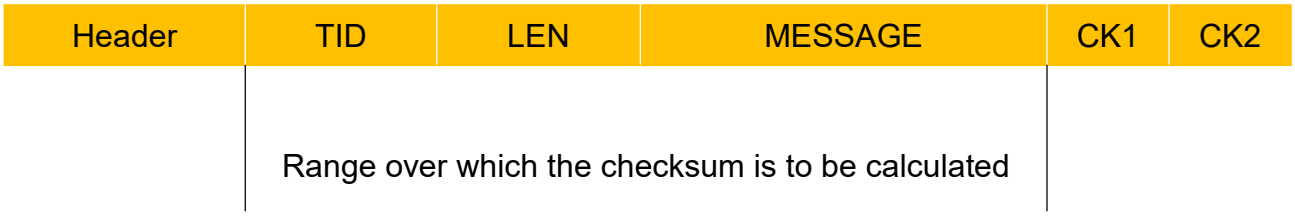

Figure 2 range for calculating checksum

Assuming that there are N bytes data(buffer[N]) for calculating, the computational formula is as follows.

```
CK1 = 0; CK2 = 0;
For(i=0;i<N;i++)
{
   CK1 = CK1 + buffer[i];
   CK2 = CK2 + CK1;
}
```

2.2 Serial command protocol

2.2.1 Frame format

YESENSE serial command protocol is different from the serial output protocol, the frame structure includes： header, command type, operation, payload data length, payload data field, checksum, as shown in the following table.

identifier	length	description
header	2 bytes	frame header is 0x59, 0x53 in little-endian
command type	1 byte	indicating which kind of command message
operation	3 bit	query or write operation
payload data length	13 bit	range is from 0 to 8191
payload data field	from 0 to 8191 bytes	valid data for command
first checksum	1 byte	checksum
second checksum	1 byte	

Command type definitions are shown in the following table.

command type	definition
product information	0x00
output baudrate	0x02
output frequency	0x03
output content	0x04
calibration parameter	0x05
function mode	0x4D

The operations are defined in the following table.

operation code	definition
0x00	query function
0x01	write without saving
0x02	write with saving

2.2.2 Payload data definition

The payload data fields corresponding to each command type are defined in the following table.

command type	sub-type	description

0x00 (product information)	0x02		software version(30 bytes)
0x02 (output baudrate)	0x01		9600 bps
	0x02		38400 bps
	0x03		115200 bps
	0x04		460800 bps
	0x05		921600 bps
	0x06		19200 bps
	0x07		57600 bps
	0x08		76800 bps
	0x09		230400 bps
0x03 (output frequency)	0x01		1Hz
	0x02		2Hz
	0x03		5Hz
	0x04		10Hz
	0x05		20Hz
	0x06		25Hz
	0x07		50Hz
	0x08		100Hz
	0x09		200Hz
	0x0A		250Hz or 400Hz
	0x0B		400Hz or 800Hz
	0x0C		1000Hz or 1600Hz
	0x0D		2000Hz or 3200Hz
	0x0E		500Hz
0x04 (output content)	Bit15	1	enable timestamp output
		0	disable timestamp output
	Bit14	1	reserved
		0	reserved
	Bit13	1	reserved
		0	reserved
	Bit12	1	reserved
		0	reserved
	Bit11	1	enable combined navigation status output

		0	disable combined navigation status output
	Bit10	1	enable timestamp output
		0	disable timestamp output
	Bit9	1	reserved
		0	reserved
	Bit8	1	reserved
		0	reserved
	Bit7	1	enable acceleration output
		0	disable acceleration output
	Bit6	1	enable angular velocity output
		0	disable angular velocity output
	Bit5	1	enable magnetic field(normalized && raw) data output
		0	disable magnetic field(normalized && raw) data output
	Bit4	1	enable euler angle output
		0	disable euler angle output
	Bit3	1	enable quaternion output
		0	disable quaternion output
	Bit2	1	enable utc time output
		0	disable utc time output
	Bit1	1	enable position output
		0	disable position output
	Bit0	1	enable velocity output
		0	disable velocity output
0x05 (calibration parameter)	0x03		gyroscope user bias(12 bytes)
	0x81		gyroscope threshold for checking static state (24 bytes)
0x4D (function mode)	0x02		fusion mode(1 byte)
	0x50		estimate gyroscope user bias(1 byte)

The following table describes payload data field for different operations and commands.

command type	input(host --> device)	output(device --> host)

0x00	query	The length of payload field is 1 byte, referring to the sub-type defined in the previous table.	The payload of response message is comprised of sub-type, length of the sub-type data and sub-type data.
0x02	query	The length of payload field is 0 byte.	The payload of response message is comprised of only sub-type(1 byte), defined in the previous table.
	write	The length of payload field is 1 byte referring to the sub-type defined in the previous table.	none
0x03	query	The length of payload field is 0 byte.	The payload of response message is comprised of only sub-type(1 byte), defined in the previous table.
	write	The length of payload field is 1 byte referring to the sub-type defined in the previous table.	The payload of response message is comprised of execution result, 1 byte.
0x04	query	The length of payload field is 0 byte.	The payload of response message is comprised of only sub-type(2 bytes), defined in the previous table.
	write	The length of payload field is 2 bytes referring to the sub-type defined in the previous table.	The payload of response message is comprised of execution result, 1 byte.
0x05	query	The length of payload field is 1 byte, refer to the sub-type defined in the previous table.	The payload of response message is comprised of sub-type, length of the sub-type data and sub-type data.
	write	The payload message is comprised of sub-type, length of the sub-type data and sub-type data.	The payload of response message is comprised of execution result, 1 byte.
0x4D	query	The length of payload field is 1 byte, refer to the sub-type defined in the previous table.	The payload of response message is comprised of sub-type and sub-type data, total 2 bytes.
	write	The length of payload field is 2 byte, including the sub-type defined in the previous table and the corresponding value.	The payload of response message is comprised of execution result, 1 byte.

YESENSE products are device role.

execution result of writing operation：0x00（successful），0xFF（fail）

This part will show more detailed definitions for more complex command type.

command type	sub-type	message definition of sub-type	comment
0x00	0x02 (software version)	model(7B) + '-' + features(5B) + '-' + version（9B, Vxx.yy.zz）+ ';' + reserved	30 bytes
0x05	0x03 (gyroscope user bias)	Bias_g_x(4B) + Bias_g_y(4B) + Bias_g_z(4B)	12 bytes and unit is dps. In the direction from host to device, the message value is singed integer and equal expected value multiply by 1000000.
	0x81 (gyroscope threshold for checking static state)	Norm_wx(4B) + Norm_wy(4B) + Norm_wz(4B) + Rms_wx(4B) + Rms_wy(4B) + Rms_wz(4B)	24 bytes and unit is dps. In the direction from host to device, the message value is singed integer and equal expected value multiply by 1000000.
0x4D	0x02 (fusion mode)	0x01：AHRS mode, 0x02：VRU mode, 0x03：IMU mode,	1 byte
	0x50 (estimate gyroscope user bias)	0x01：estimate self	

Bias_g_x is the abbreviation for gyroscope user bias of x axis.

Bias_g_y is the abbreviation for gyroscope user bias of y axis.

Bias_g_z is the abbreviation for gyroscope user bias of z axis.

Norm_wx and Rms_wx are abbreviations for the absolute and rms value of gyroscope's x axis.

Norm_wy and Rms_wy are abbreviations for the absolute and rms value of gyroscope's y axis.

Norm_wz and Rms_wz are abbreviations for the absolute and rms value of gyroscope's z axis.

Note：The command of configuring the serial baudrate has on result message.

2.2.3 Checksum

A whole data frame needs to append checksum to the tail of payload. The calculated range starts from command type to the last byte of message, as shown in the following figure, and the calculation formula is shown in the following example.

header	command type	op(2:0)	len(15:3)	message	CK1	CK2
0x59 0x53	Range over which the checksum is to be calculated					

Assuming that there are N bytes data(buffer[N]) for calculating, the computational formula is as follows.

```
CK1 = 0；CK2 = 0；
For(i=0;i<N;i++)
{
    CK1 = CK1 + buffer[i];
    CK2 = CK2 + CK1;
}
```

3 Reference

3.1 Examples

3.1.1 Example message of serial output protocol

The following data is an instance of a whole output message.

```
0x59 ,0x53 ,0x40 ,0x92 ,0x58 ,0x10 ,0x0c ,0x3b ,0x21 ,0xfe ,0xff ,0x89 ,0x2c ,0xfe ,0xff ,
0x59 ,0x9c ,0x6a ,0xff ,0x20 ,0x0c ,0xd1 ,0xa2 ,0x02 ,0x00 ,0xfa ,0xb1 ,0x05 ,0x00 ,0x38 ,
0xd0 ,0x00 ,0x00 ,0x30 ,0x0c ,0x20 ,0xff ,0xf1 ,0x07 ,0xd0 ,0xa0 ,0xea ,0xf4 ,0xc0 ,0x8e ,
0xc6 ,0xef ,0x31 ,0x0c ,0xb4 ,0x08 ,0x02 ,0x00 ,0xa2 ,0x29 ,0xfd ,0xff ,0xb8 ,0xd8 ,0xfb ,
0xff ,0x40 ,0x0c ,0xcb ,0xe4 ,0xf4 ,0xff ,0xb1 ,0xbe ,0x09 ,0x00 ,0x30 ,0x32 ,0xb6 ,0xf6 ,
0x41 ,0x10 ,0xd4 ,0x31 ,0x03 ,0x00 ,0x4d ,0xec ,0xff ,0xff ,0x86 ,0xe5 ,0xff ,0xff ,0xa9 ,
0x14 ,0xf1 ,0xff ,0x12 ,0xf3
```

Each corresponding part of serial output protocol in this example are explained as following.

no	content	description
1	0x59 ,0x53 ,	frame header, little-endian
2	0x40 ,0x92 ,	frame sequence number, little-endian
3	0x58 ,	message length
4	0x10 , … , 0xff ,	valid message, little-endian
5	0x12 ,0xf3	checksum, little-endian

How to decode the payload data field of this example is demonstrated as following.

```
0x10 ,0x0C , 0x3b ,0x21 ,0xfe ,0xff ,0x89 ,0x2c ,0xfe ,0xff , 0x59 ,0x9c ,0x6a ,0xff
```

0x10(DATA ID): indicating acceleration message

0x0C(LEN): indicating acceleration message has twelve bytes

The left data are the content of acceleration that each four bytes constitute an signed integer value and then the final data is the result of the signed integer multiplied by factor 0.000001. Such as the first set four bytes data is 0x3B 0x21 0xFE 0xFF in little-endian and corresponding signed integer data is `0xFFFE213B`(-122565 in decimal), that's to say the acceleration value of x axis is the result of 122565* 0.000001(-0.122565) m/s².

```
0x20 ,0x0C , 0xd1 ,0xa2 ,0x02 ,0x00 ,0xfa ,0xb1 ,0x05 ,0x00 ,0x38 , 0xd0 ,0x00 ,0x00
```

0x20(DATA ID): indicating angular velocity message

0x0C(LEN): indicating angular velocity message has twelve bytes

The left data are the content of angular velocity that each four bytes constitute an signed integer value and then the final data is the result of the signed integer multiplied by factor 0.000001.

Such as the second set four bytes data is 0xFA 0xB1 0x05 0x00 in little-endian and corresponding signed integer data is 0x0005B1FA(373242 in decimal), that's to say the angular velocity value of y axis is the result of 373242 * 0.000001(0.373242) deg/s.

```
0x30 ,0x0C , 0x20 ,0xff ,0xf1 ,0x07 ,0xd0 ,0xa0 ,0xea ,0xf4 ,0xc0 ,0x8e , 0xc6 ,0xef
```

0x30(DATA ID): indicating magnetic field normalized value message

0x0C(LEN): indicating magnetic field normalized value message has twelve bytes

The left data are the content of magnetic field normalized value that each four bytes constitute an signed integer value and then the final data is the result of the signed integer multiplied by factor 0.000001. Such as the third set four bytes data is 0xC0 0x8E 0xC6 0xEF in little-endian and corresponding signed integer data is 0xEFC68EC0(-272200000 in decimal), that's to say the magnetic field normalized value of z axis is the result of -272200000 * 0.000001(-272.2).

```
0x31 ,0x0C , 0xb4 ,0x08 ,0x02 ,0x00 ,0xa2 ,0x29 ,0xfd ,0xff ,0xb8 ,0xd8 ,0xfb , 0xff
```

0x31(DATA ID): indicating magnetic field raw value data message

0x0C(LEN): indicating magnetic field raw value data message has twelve bytes

The left data are the content of magnetic field raw data that each four bytes constitute an signed integer value and then the final data is the result of the signed integer multiplied by factor 0.001. Such as the first set four bytes data is0xB4 0x08 0x02 0x00 in little-endian and corresponding signed integer data is 0x000208B4(133300in decimal), that's to say the magnetic field raw value of x axis is the result of 133300* 0.001(133.30) mGauss.

```
0x40 ,0x0C , 0xcb ,0xe4 ,0xf4 ,0xff ,0xb1 ,0xbe ,0x09 ,0x00 ,0x30 ,0x32 ,0xb6 ,0xf6
```

0x40(DATA ID): indicating euler angle message

0x0C(LEN): indicating euler angle message has twelve bytes

The left data are the content of euler angle that each four bytes constitute an signed integer value and then the final data is the result of the signed integer multiplied by factor 0.000001. Such as the third set four bytes data is 0x30 0x32 0xB6 0xF6 in little-endian and corresponding signed integer data is 0xF6B63230 (-155831760 in decimal), that's to say the yaw value is the result of -155831760 * 0.000001(-155.83176) deg.

```
0x41 ,0x10 , 0xd4 ,0x31 ,0x03 ,0x00 ,0x4d ,0xec ,0xff ,0xff ,0x86 ,0xe5 ,0xff ,0xff ,0xa9 ,
0x14 ,0xf1 ,0xff
```

0x41(DATA ID): indicating quaternion message

0x10(LEN): indicating quaternion message has sixteen bytes

The left data are the content of quaternion that each four bytes constitute an signed integer value and then the final data is the result of the signed integer multiplied by factor 0.000001. Such as the fourth set four bytes data is 0xA9 0x14 0xF1 0xFF in little-endian and corresponding signed integer data is 0xFFF114A9 (-977751 in decimal), that's to say the q3 value is the result of -977751* 0.000001(-0. 977751).

3.1.2 Command examples

command description	command/result(hex)	
Configure output frequency to 20Hz without saving	command	59 53 03 09 00 05 11 2C
	result for successful	59 53 03 09 00 00 0c 27
	result for fail	59 53 03 09 00 ff 0b 26
Configure output frequency to 20Hz with saving	command	59 53 03 0A 00 05 12 2F
	result for successful	59 53 03 0a 00 00 0d 2a
	result for fail	59 53 03 0a 00 ff 0c 29
Configure output content to none without saving	command	59 53 04 11 00 00 00 15 58
	result for successful	59 53 04 09 00 00 0d 2b
	result for fail	59 53 04 09 00 ff 0c 2a
Configure output content to only acceleration without saving	command	59 53 04 11 00 80 00 95 58
	result for successful	59 53 04 09 00 00 0d 2b
	result for fail	59 53 04 09 00 ff 0c 2a
Configure output content to only angular velocity without saving	command	59 53 04 11 00 40 00 55 D8
	result for successful	59 53 04 09 00 00 0d 2b
	result for fail	59 53 04 09 00 ff 0c 2a
Configure output content to acceleration, angular velocity, euler angle and quaternion without saving	command	59 53 04 11 00 D8 00 ED 08
	result for successful	59 53 04 09 00 00 0d 2b
	result for fail	59 53 04 09 00 ff 0c 2a
Configure output content to acceleration, angular velocity, magnetic field daa,euler angle and quaternion with saving	command	59 53 04 12 00 F8 00 0E 4C
	result for successful	59 53 04 0a 00 00 0e 2e
	result for fail	59 53 04 0a 00 ff 0d 2d
Configure output content to only euler angle with saving	command	59 53 04 12 00 10 00 26 7C
	result for successful	59 53 04 0a 00 00 0e 2e
	result for fail	59 53 04 0a 00 ff 0d 2d
Configure output baudrate to 115200	command	59 53 02 09 00 03 0E 26

bps without saving	result for successful	none
	result for fail	none
Configure output baudrate to 460800bps without saving	command	59 53 02 09 00 04 0F 27
	result for successful	none
	result for fail	none
Configure output baudrate to 460800bps with saving	command	59 53 02 0A 00 04 10 2A
	result for successful	none
	result for fail	none

Note：

Please be careful when you'd like to configuring baudrate, because the required baudrate has a relationship with output content and output frequency. If the baudrate is configured too low, the output message will not match the configuration of output content or output frequency.

Generally, the minimum value of baudrate can be estimated as follows.

Step 1: determine the expected output content to calculate how many bytes per frame and the number is marked as N bytes.

Step 2: determine the expected output frequency, marked as f Hz.

Step 3: calculate the appropriate baudrate according to step 1 and 2. Baudrate can be calculated by the following formula, baudrate = N * 10 * f / 0.8.

N is from Step 1, f is from Step 2.

4 Revisions

Revision	Date	By	Changes
1.0	2018-10-30	Z. L.	• Initial release
2.3	2023-10-27	F. L.	• Update latest protocol